Divide & Conquer Algorithm Visualizer

Course: Design and Analysis of Algorithms **Roll Numbers:** 24k-1033, 24k-0945, 24k-0604

a. Abstract

This project explores the implementation and visualization of various Divide-and-Conquer algorithms. We implemented fundamental algorithms including finding minimum/maximum, exponentiation, counting inversions, and Quicksort analysis. Furthermore, we focused heavily on two advanced algorithms: the Karatsuba Integer Multiplication and the Closest Pair of Points (2D). To demonstrate their efficiency, we developed an interactive graphical user interface (GUI). The system accepts randomly generated datasets (size > 100) from text files and visually demonstrates the step-by-step execution of these algorithms, validating their correctness and benchmarking their performance against brute-force alternatives.

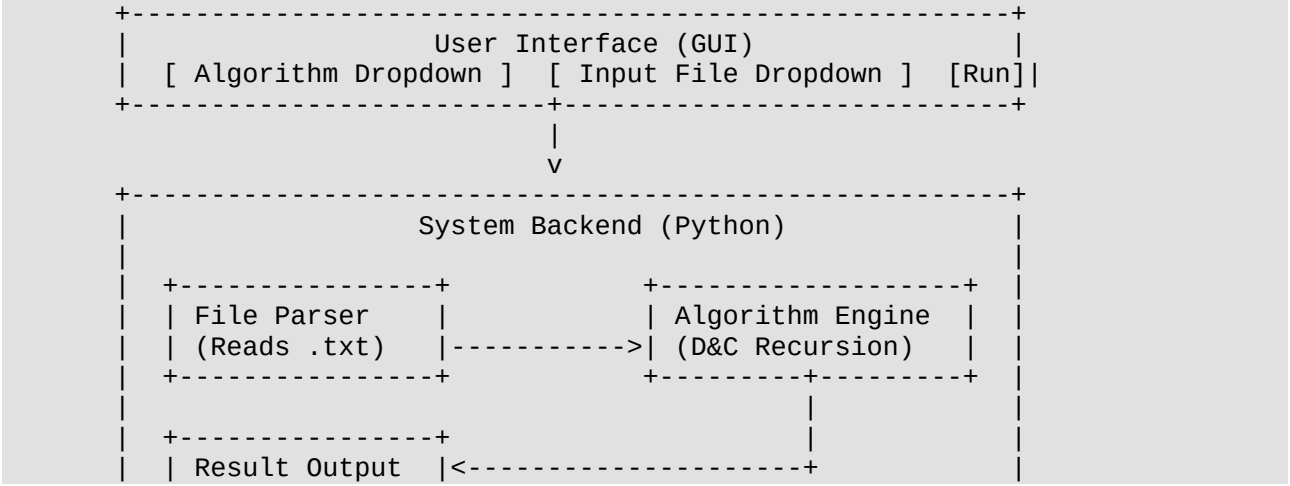
b. Introduction

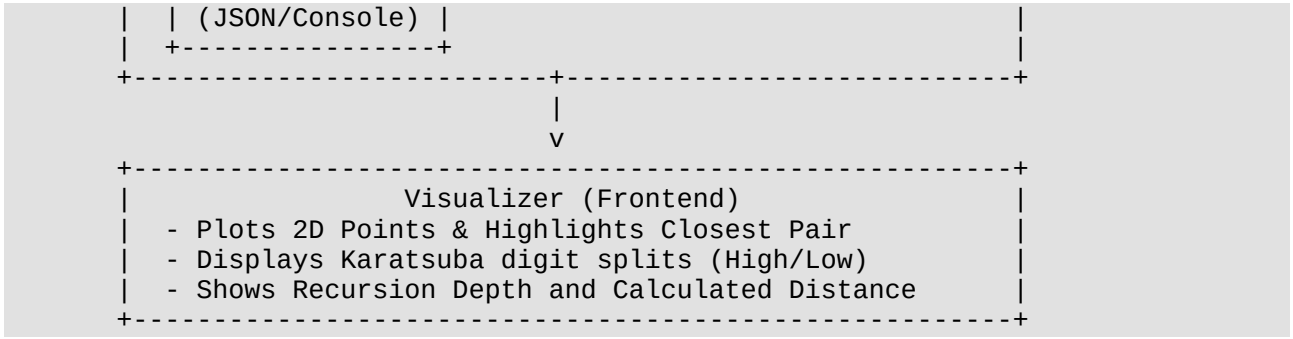
The Divide-and-Conquer paradigm is a fundamental algorithmic design technique that solves a problem by recursively breaking it down into two or more sub-problems of the same or related type, until these become simple enough to be solved directly. The solutions to the sub-problems are then combined to give a solution to the original problem.

For this project, we first solved several theoretical and practical exercises from Anany Levitin's *Introduction to the Design and Analysis of Algorithms* and Jon Kleinberg's *Algorithm Design*. This included deriving recurrence relations for Min/Max (O(n)), Exponentiation (O(log n)), and analyzing Quicksort (Worst-case: $O(n^2)$, Best-case: O(n log n)). We then implemented two complex D&C algorithms—Karatsuba Multiplication (O(n^1.585)) and Closest Pair of Points (O(n log n))—and built a system to process large datasets and visualize their internal mechanics.

c. Your Proposed System

Our proposed system is a web-based visualizer built to process and display the execution of the Closest Pair and Karatsuba algorithms.





System Discussion: The system starts with a user-friendly interface where users select an algorithm (Karatsuba or Closest Pair) and a specific text file. The Python backend parses the numerical data. For the Closest Pair, it plots the coordinates and visually tracks the recursive splits until the closest pair is highlighted. For Karatsuba, it parses massive integers, displays the split at (10^{\text{mid}}), and shows the recursive high/low calculations.

d. Experimental Setup

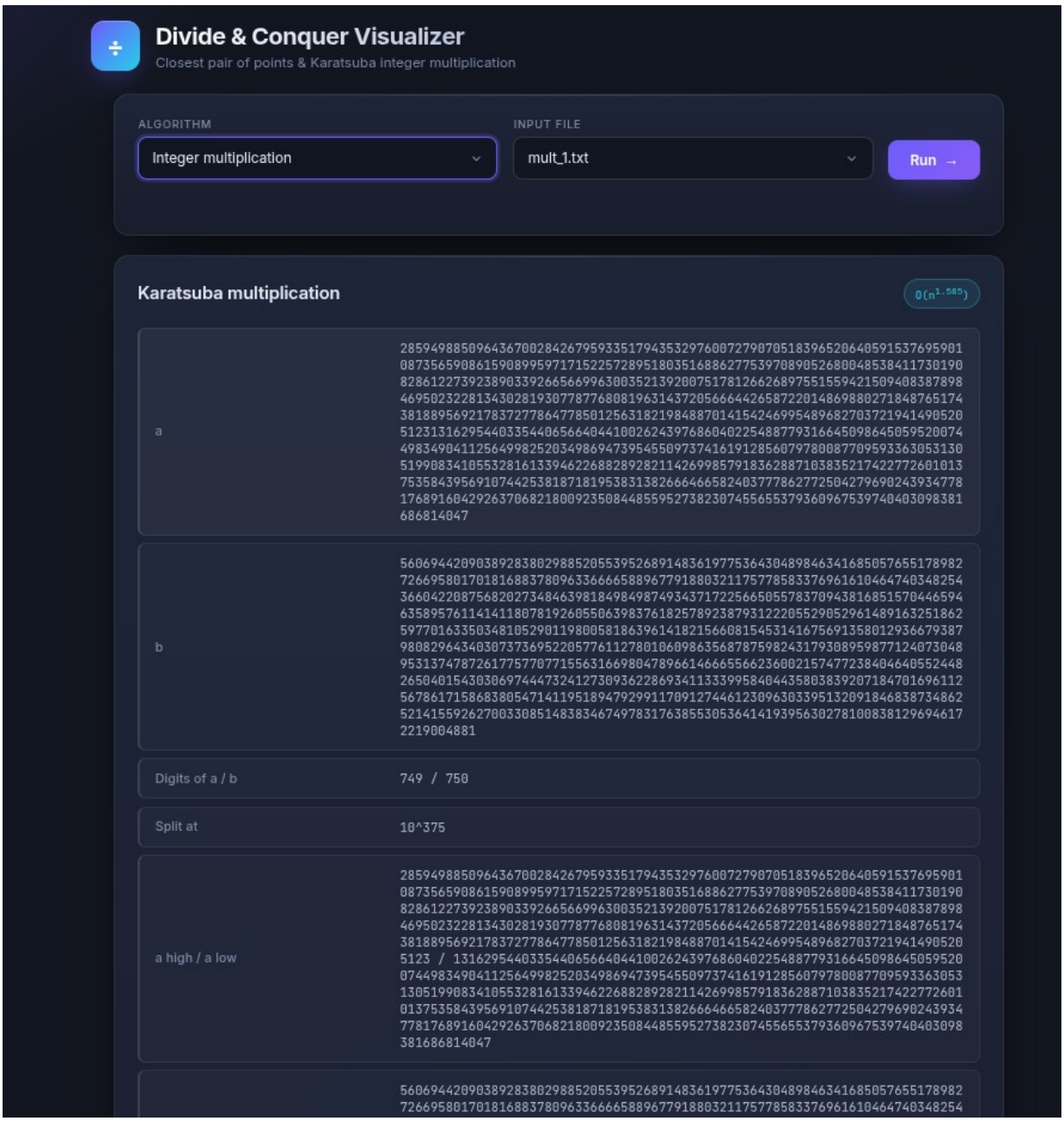
To benchmark the algorithms, we generated random datasets and saved them to text files.

- **Closest Pair Datasets:** 10 files (`closest_1.txt` to `closest_10.txt`). Each file contains coordinates of randomly generated floating-point numbers. The input size was set to >100 (e.g., `closest_3.txt` contains 400 points).
- **Integer Multiplication Datasets:** 10 files (`mult_1.txt` to `mult_10.txt`). Each file contains two very large integers generated randomly with lengths >100 digits.
- **Environment:** The algorithms were implemented in Python, and the UI was rendered via a local web server. Results were also printed to the console to verify algorithmic correctness against Python's native arbitrary-precision arithmetic (`a * b == result`).

e. Results and Discussion

The system successfully executed and visualized both algorithms across all 20 datasets.

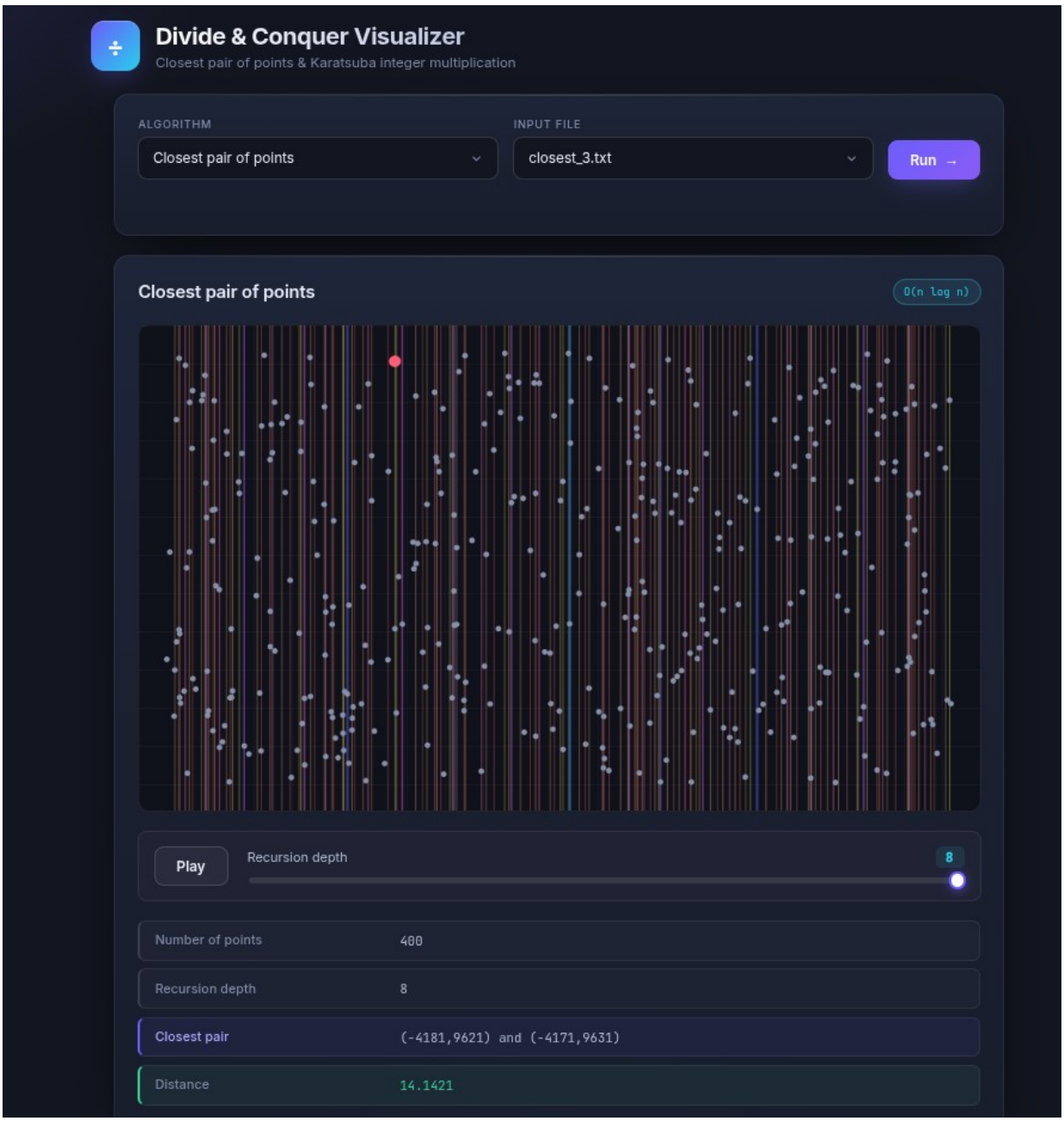
1. Karatsuba Integer Multiplication:



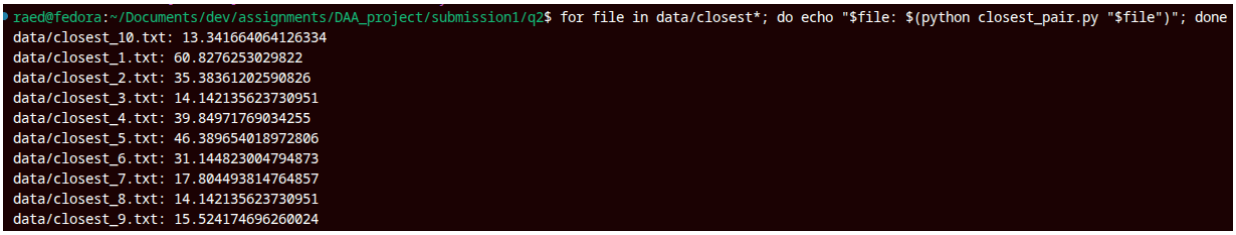
- *Caption: UI showing `mult_1.txt` with 749 and 750 digits. The split at 10^375 is shown, breaking the numbers into high and low parts.*

Terminal verification showing `True` for all 10 files, proving our D&C implementation perfectly matches Python's native multiplication.

2. Closest Pair of Points:



UI showing `closest_3.txt` with 400 points. The closest pair is highlighted in red, with a calculated distance of 14.1421 at recursion depth 8.



Terminal output showing the minimum distance found for all 10 datasets. The distances range from ~13.34 to ~60.82.

Discussion: The visualizer proves that the Divide-and-Conquer Closest Pair algorithm efficiently narrows down the search space, avoiding the (O(n^2)) brute-force comparison. By sorting and dividing the plane, it finds the minimum distance in (O(n \log n)) time. Similarly, Karatsuba reduces the complexity of large integer multiplication from (O(n^2)) to (O(n^{1.585})) by reducing the number of recursive multiplications from four to three.

f. Conclusion

In this project, we successfully designed and implemented a comprehensive suite of Divide-and-Conquer algorithms. The theoretical recurrence relations for standard algorithms were verified. The interactive visualizer built for Karatsuba Multiplication and the Closest Pair of Points demonstrates the practical efficiency of the D&C paradigm. By testing on 20 large datasets, we confirmed that the algorithms are robust, accurate, and significantly faster than their brute-force counterparts. The UI provides excellent pedagogical insight into how sub-problems are divided and conquered in real-time.

g. References

1. Levitin, A. (2012). *Introduction to the Design and Analysis of Algorithms* (3rd ed.). Pearson.
2. Kleinberg, J., & Tardos, É. (2005). *Algorithm Design* (1st ed.). Pearson.
3. Python Software Foundation. (2024). Python Language Reference. Available at: https://www.python.org/